CSM GLEAMING 2 Meta

COMPUTER SCIENCE MENTORS 61A

November 9, 2026 – November 13, 2026

1 More Gleam!

These questions are adapted from past CS 61A exams related to linked lists, trees, and/or Scheme.

Here is some commonly used list syntax:

- `[]` is empty.
- `[first, ..rest]` splits a non-empty list.
- `[first, ..rest]` also constructs a new list.
- `#(x, y)` is a two-element tuple.

Here are some commonly used built-in list functions:

- `list.append(xs, ys)` joins two lists.
- `list.map(xs, f)` applies `f` to each value.
- `list.filter(xs, keep)` keeps values satisfying `keep`.
- `list.length(xs)` returns a list's length.

You can find other useful built-in functions in Gleam here:
https://gleam-stdlib.hexdocs.pm/gleam/list.html
https://gleam-stdlib.hexdocs.pm/gleam/int.html
https://gleam-stdlib.hexdocs.pm/gleam/result.html

Created by Ana Abell, Terry Hsu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. *Adapted from CS 61A Fall 2024 Final Q7 (A Pair of Schemes)*

   (a) Implement `all_pairs`, which takes a two-argument function `f` and a list `s`. It returns `True` if `f(x, y)` returns `True` for every pair of adjacent elements `x` and `y` in `s`. The function `inc` returns whether `x + 1` equals `y`.

```
pub fn inc(x: Int, y: Int) -> Bool {
  x + 1 == y
}

pub fn all_pairs(f: fn(a, a) -> Bool, s: List(a)) -> Bool {
  case s {
    [x, y, ..rest] -> _____________ && all_pairs(f, _____________)
    _ -> True
  }
}
assert all_pairs(inc, [3, 4, 5, 6, 7, 8]) == True
assert all_pairs(inc, [3, 4, 5, 8, 7, 8]) == False
assert all_pairs(inc, [3, 4, 6, 7]) == False
assert all_pairs(inc, [3]) == True
assert all_pairs(inc, []) == True
```

```
pub fn all_pairs(f: fn(a, a) -> Bool, s: List(a)) -> Bool {
  case s {
    [x, y, ..rest] -> f(x, y) && all_pairs(f, [y, ..rest])
    _ -> True
  }
}
```

Adjacent pairs overlap: in `[3, 4, 5]`, the pairs are `(3, 4)` and `(4, 5)`, so `4` belongs to both. After checking `x` and `y`, the recursive call must start at `y`, not at the element after it. Writing `all_pairs(f, rest)` would skip every other pair, and would incorrectly return `True` on `[3, 4, 6, 7]`. The pattern `[x, y, ..rest]` only matches lists with at least two elements. Every other list (empty or one element) has no pairs, so the condition holds for all of them and the `_` case returns `True`. `&&` only evaluates its right side when its left side is `True`, so `all_pairs` stops at the first pair that fails.

(b) Implement `show_pairs`, which takes a list `s` and returns a list of all pairs of adjacent elements in `s`, in order. Each pair is a tuple.

```
pub fn show_pairs(s: List(a)) -> List(#(a, a)) {
  case s {
    [x, y, ..rest] -> [__________, ..show_pairs(________________)]
    _ -> []
  }
}
assert show_pairs([3, 5, 7, 9, 11, 13])
  == [#(3, 5), #(5, 7), #(7, 9), #(9, 11), #(11, 13)]
assert show_pairs([1, 2]) == [#(1, 2)]
assert show_pairs([1]) == []
```

```
pub fn show_pairs(s: List(a)) -> List(#(a, a)) {
  case s {
    [x, y, ..rest] -> [#(x, y), ..show_pairs([y, ..rest])]
    _ -> []
  }
}
```

This has the same structure as `all_pairs`, but instead of combining each pair's result with `&&`, it collects the pairs themselves into a list. As before, the recursive call starts at `y` because adjacent pairs overlap. A list with $n \geq 1$ elements has $n - 1$ adjacent pairs, so the result is always one element shorter than `s`.

(c) Implement `all_pairs_calls`, which takes a two-argument function `f` and a list `s`. It returns one *delayed call* (a function that takes no arguments) for each pair of adjacent elements `x` and `y` in `s`, which calls `f(x, y)` when it is called. Then implement `run_all`, which calls each delayed call in a list and returns whether all of them returned `True`. You may use `show_pairs`.

```
pub fn all_pairs_calls(f: fn(a, a) -> Bool, s: List(a)) -> List(fn()
    -> Bool) {
  list.map(show_pairs(s), fn(pair) { ____________________________________ })
}

pub fn run_all(calls: List(fn() -> Bool)) -> Bool {
  list.all(calls, _________________________)
}
assert list.length(all_pairs_calls(inc, [3, 4, 5, 6, 7, 8])) == 5
assert run_all(all_pairs_calls(inc, [3, 4, 5, 6, 7, 8])) == True
assert run_all(all_pairs_calls(inc, [3, 4, 5, 8, 7, 8])) == False
assert run_all([]) == True
```

```
pub fn all_pairs_calls(f: fn(a, a) -> Bool, s: List(a)) -> List(fn()
    -> Bool) {
  list.map(show_pairs(s), fn(pair) { fn() { f(pair.0, pair.1) } })
}

pub fn run_all(calls: List(fn() -> Bool)) -> Bool {
  list.all(calls, fn(call) { call() })
}
```

Writing `f(pair.0, pair.1)` in the first blank would call `f` right away and put a `Bool` in the list. Wrapping it as `fn(){ f(pair.0, pair.1)}` creates a function that will make that call later. So `all_pairs_calls` does none of the actual checking; it only describes what to check. The fields of a tuple are accessed by position with `.0` and `.1`. We could also have unpacked the tuple first with `let #(x, y)= pair` and then returned `fn(){ f(x, y)}`. `run_all` is where the calls actually happen: `list.all` calls the function `fn(call){ call()}` on each delayed call. Like `&&`, `list.all` stops at the first `False`, so the later calls are never made. Together, `run_all(all_pairs_calls(f, s))` gives the same answer as `all_pairs(f, s)`. The difference is when the work happens. This is the same idea as the original Scheme question, which built the expression `(and (inc 3 4)(inc 4 5)...)` and passed it to `eval` later. Read `List(fn()-> Bool)` from the inside out: each element is a function with no arguments that returns a `Bool`, and the outer list holds those functions. The inner `fn(){ ... }` is a closure: it remembers `f` and `pair` now, but calls `f` only when `call()` runs later. A helpful check is to compare the types: without the inner function, `list.map` would produce `List(Bool)`, which is not the declared return type.

2. *Adapted from CS 61A Fall 2022 Final Q4b (Advent of Code)*

   Implement `find_pair`, which takes a list of integers `s` and returns the two elements of `s` that sum to 2020, in the order that they appear in `s`. Assume that exactly one pair of elements sums to 2020. You may call `list.contains(s, x)`, which returns whether `s` contains `x`.

```
pub fn find_pair(s: List(Int)) -> List(Int) {
  case s {
    [] -> []
    [x, ..rest] ->
      case ____________________________________ {
        True -> ___________________
        False -> ___________________
      }
  }
}
assert find_pair([979, 1721, 366, 299, 675, 1456]) == [1721, 299]
assert find_pair([2000, 20]) == [2000, 20]
assert find_pair([1, 1010, 3, 1010]) == [1010, 1010]
```

```
pub fn find_pair(s: List(Int)) -> List(Int) {
  case s {
    [] -> []
    [x, ..rest] ->
      case list.contains(rest, 2020 - x) {
        True -> [x, 2020 - x]
        False -> find_pair(rest)
      }
  }
}
```

   Instead of trying every pair, we only ask one question about each element `x`: does its partner `2020 - x` appear later in the list? If so, we already know both numbers, so we can return `[x, 2020 - x]` without finding where the partner is. We only search `rest` rather than all of `s`. Any partner earlier in the list would have been found when we were at that earlier element, so by the time we reach `x`, nothing before it can be part of the answer. Searching `rest` also means `x` can never be paired with itself, which is why the last assert works even though 1010 + 1010 is 2020. (The original exam guaranteed that `s` contains no 1010, since its Python version could pair an element with itself.) The `[] -> []` case is never reached when a pair exists, but Gleam requires every case to be handled. This takes quadratic time: each call to `list.contains` can scan the rest of the list, and there can be one call per element. When walking through the nested `case`, have students name the values at each level: the outer match binds `x` and `rest`; the inner match branches on a `Bool`. A `case` is an expression in Gleam, so the inner one supplies the value used by the outer branch. The guard `if x == y` pattern used in other Gleam questions has the same role of refining when a branch matches.

3. *Adapted from CS 61A Fall 2014 Midterm 2 Q3b (This One Goes to Eleven)*

Implement `no_eleven`, which takes a non-negative integer `n`. It returns a list of all distinct lists of length `n` that contain only `1`s and `6`s, in which two `1`s never appear next to each other.

```
pub fn no_eleven(n: Int) -> List(List(Int)) {
  case n {
    0 -> _________
    1 -> [[6], [1]]
    _ ->
      list.append(
        list.map(_____________, fn(s) { [6, ..s] }),
        list.map(_____________, fn(s) { _______________ }),
      )
  }
}
assert no_eleven(0) == [[]]
assert no_eleven(2) == [[6, 6], [6, 1], [1, 6]]
assert no_eleven(3)
  == [[6, 6, 6], [6, 6, 1], [6, 1, 6], [1, 6, 6], [1, 6, 1]]
assert no_eleven(4)
  == [[6, 6, 6, 6], [6, 6, 6, 1], [6, 6, 1, 6], [6, 1, 6, 6],
      [6, 1, 6, 1], [1, 6, 6, 6], [1, 6, 6, 1], [1, 6, 1, 6]]
```

```
pub fn no_eleven(n: Int) -> List(List(Int)) {
  case n {
    0 -> [[]]
    1 -> [[6], [1]]
    _ ->
      list.append(
        list.map(no_eleven(n - 1), fn(s) { [6, ..s] }),
        list.map(no_eleven(n - 2), fn(s) { [1, 6, ..s] }),
      )
  }
}
```

Split the answers by their first element. If a list starts with `6`, nothing constrains what comes next, so the rest can be any valid list of length $n-1$. If a list starts with `1`, the next element can't also be `1`, so it must be `6`. After that `6`, the rest can again be anything valid, now of length $n-2$. That is why the second group prepends two elements, `[1, 6, ..s]`, and recurses on `n - 2`. The base case for `0` is `[[]]`, not `[]`. There is exactly one list of length 0, the empty list, and it contains no `11`. Returning `[]` would claim there are no valid lists, and since `list.map` over an empty list gives an empty list, every larger `n` would also return `[]`. We need a separate base case for `1` because `no_eleven(n - 2)` would otherwise be called with `-1`. A list of length 1 can start with `1` without being followed by a `6`. The number of lists returned follows the Fibonacci sequence: 1, 2, 3, 5, 8, ..., because the count for n is the count for $n-1$ plus the count for $n-2$.

4. *Adapted from CS 61A Spring 2018 Midterm 2 Q4b (Sequences)*

   Implement `combo`, which takes two lists of digits `a` and `b`. It returns the smallest list of digits that contains all of the digits of `a` in order, as well as all of the digits of `b` in order. A list is smaller than another if it is shorter, or if they have the same length and it comes first when compared digit by digit. You may use `smaller`, which returns the smaller of two lists.

```
pub fn smaller(s: List(Int), t: List(Int)) -> List(Int) {
  // Implementation omitted
}

pub fn combo(a: List(Int), b: List(Int)) -> List(Int) {
  case a, b {
    [], _ -> ____
    _, [] -> a
    [x, ..ar], [y, ..br] if x == y -> _____________________
    [x, ..ar], [y, ..br] ->
      smaller(_____________________, _____________________)
  }
}
assert combo([5, 3, 1], [4, 3, 2]) == [4, 5, 3, 1, 2]
assert combo([5, 3, 1], [4, 3, 2, 1]) == [4, 5, 3, 2, 1]
assert combo([1, 2, 3, 4], [9, 1, 2, 3]) == [9, 1, 2, 3, 4]
assert combo([], [3, 2, 1]) == [3, 2, 1]
```

```
pub fn combo(a: List(Int), b: List(Int)) -> List(Int) {
  case a, b {
    [], _ -> b
    _, [] -> a
    [x, ..ar], [y, ..br] if x == y -> [x, ..combo(ar, br)]
    [x, ..ar], [y, ..br] ->
      smaller([x, ..combo(ar, b)], [y, ..combo(a, br)])
  }
}
```

The answer has to begin with either the first digit of `a` or the first digit of `b`; any other digit would be wasted. When the two first digits are equal, a single digit can serve both lists at once, so we use it once and continue with the rest of both lists. When the first digits are different, we try both options. If the answer starts with `x`, that digit only accounts for the start of `a`, so we still need all of `b` afterward: `combo(ar, b)`, not `combo(ar, br)`. The other option is symmetric. `smaller` then picks the better of the two. When one list is empty, the answer is just the other list, since every digit of it has to appear and nothing else is needed. This is tree recursion with two branches, so it takes exponential time in the worst case. The original question used integers like `531` instead of lists of digits, and it started from the last digit using `% 10` and `// 10`. Starting from the front of a list gives the same answers. In the integer version, "smallest" compared numbers, which is why `smaller` first compares lengths and then digits. To make the two recursive choices concrete, try `combo([5, 3], [4, 3])`: choosing 5 next leaves `combo([3], [4, 3])`; choosing 4 next leaves `combo([5, 3], [3])`. Both candidates must preserve each input's order, and `smaller` compares the completed candidates by length first, then lexicographically. This is a useful point to distinguish the problem's custom ordering from ordinary numeric comparison.

5. *Adapted from CS 61A Fall 2023 Final Q6 (How to Get Promoted)*

   (a) Implement `promote`, which takes a function `f` and a list `s`. It returns a list containing all of the elements of `s`, with the elements for which `f` returns `True` at the front. Otherwise, the elements stay in the same order as in `s`.

```
pub fn promote(f: fn(a) -> Bool, s: List(a)) -> List(a) {
  __________(list.filter(s, __________), list.filter(s, __________))
}
assert promote(fn(x) { x % 2 == 0 }, [1, 2, 3, 4, 5, 6, 7])
  == [2, 4, 6, 1, 3, 5, 7]
assert promote(fn(x) { x % 2 == 1 }, [1, 2, 3, 4, 5, 6, 7])
  == [1, 3, 5, 7, 2, 4, 6]
assert promote(fn(x) { x > 100 }, [1, 2, 3]) == [1, 2, 3]
```

```
pub fn promote(f: fn(a) -> Bool, s: List(a)) -> List(a) {
  list.append(list.filter(s, f), list.filter(s, fn(x) { !f(x) }))
}
```

The result has two parts: the elements for which `f` is true, then the elements for which it is false. Each part is a `list.filter` of `s`, and `list.filter` keeps elements in their original order, so joining the two parts with `list.append` keeps the order within each group. The second filter needs a function that is true exactly when `f` is false. Gleam has no built-in way to negate a function, so we write one: `fn(x){ !f(x)}`. The standard library also has `list.partition(s, f)`, which returns both groups at once as a tuple `#(trues, falses)`. With it, we could have written `let #(yes, no)= list.partition(s, f)` and then returned `list.append(yes, no)`. This goes through `s` only once instead of twice.

(b) Implement `bigger_first`, which takes a non-empty list `s`. It returns a list with all of the elements of `s` that are larger than the first element at the front, followed by all of the elements that are smaller than or equal to the first element. Each group keeps its original order. You may use `promote` from the previous part and the functions below. **You may not write** `fn`, `case`, `[`, or `]` in the blank.

```
pub fn curry(f: fn(a, b) -> c) -> fn(a) -> fn(b) -> c {
  fn(x) { fn(y) { f(x, y) } }
}

pub fn curry_call(f: fn(x, y) -> r) -> fn(fn(z) -> x, fn(y) -> z) ->
    fn(y) -> r {
  fn(x, g) { fn(y) { f(x(g(y)), y) } }
}

pub fn less_than(x: Int, y: Int) -> Bool {
  x < y
}

pub fn first(s: List(Int)) -> Int {
  let assert [x, ..] = s
  x
}

pub fn bigger_first(s: List(Int)) -> List(Int) {
  ____________________________________________(s)
}
assert bigger_first([3, 1, 4, 1, 5, 9, 2, 6]) == [4, 5, 9, 6, 3, 1, 1,
    2]
assert bigger_first([5, 1, 2]) == [5, 1, 2]
assert bigger_first([1, 2, 3]) == [2, 3, 1]
```

```
pub fn bigger_first(s: List(Int)) -> List(Int) {
  curry_call(promote)(curry(less_than), first)(s)
}
```

Without the restrictions, the answer would be `promote(fn(y){ first(s)< y }, s)`. The difficulty is that the predicate depends on `s`, so `s` has to be used twice: once to build the predicate and once as the list to promote. `curry_call` does exactly this. `curry_call(f)(x, g)` is a function of `y` that computes `f(x(g(y)), y)`, passing `y` both through `x` and `g` and directly. Matching this against what we want with `y` as `s`, `f` is `promote`, `g(s)` should be `first(s)`, and `x(first(s))` should be the predicate `fn(y){ first(s)< y }`. That is what `curry(less_than)` gives: `curry(less_than)(a)` is `fn(y){ a < y }`. The types can guide you here too. `curry_call(promote)` expects some `fn(z)-> fn(Int)-> Bool` and some `fn(List(Int))-> z`. The only provided function that takes a `List(Int)` is `first`, which fixes `z` as `Int`, and the only way to get an `fn(Int)-> fn(Int)-> Bool` is `curry(less_than)`. The first element is never bigger than itself, so it always ends up at the front of the second group.

2 Interpreters

Interpreters can seem daunting to design, since an interpreter is a program that can carry out any possible computation depending on its input. Yet many share an elegant structure built from two mutually recursive functions: `eval`, which takes an expression and an environment and returns a value, and `apply`, which takes a function and a list of argument values and returns a value. An environment is a mapping from names to values, and the two functions are defined in terms of each other because applying a user-defined function means evaluating its body in a new environment that binds the parameters to the arguments, while evaluating an expression may involve applying one or more functions.

`eval` handles each kind of expression differently: numbers evaluate to themselves, names are looked up in the environment, and calls evaluate their operator and operands before handing the results to `apply`. Special forms such as `case` are the exception, since they do not evaluate every subexpression and only run the branch that matches. Recursion stops at the simple cases (numbers and name lookups) and at primitive operations like `+`, which `apply` computes directly without evaluating a body. This back-and-forth is the eval-apply loop, and it continues until the whole program has been reduced to a single value.

Gleam's approach to types and pattern matching suits this design well. If expressions are represented by a type with one variant per form, such as `N`, `Var`, and `Binary`, then `eval` is a `case` expression with one branch per variant, and the compiler will refuse to compile it if you forget a form. To see how all of this plays out, we can count the calls to `eval` and `apply` as a program runs.

Sometimes it helps to visualize how an interpreter works by counting the calls to eval and apply as we run our code. There are some rules to it. Essentially, every time we encounter a new block of code whose value we're not sure of, we need to evaluate it. In a basic function call, we need to evaluate the function, then evaluate the operands, and only then can we apply the function to its operands. The next page has a more detailed example.

Something worth noting (also shown as a comment in the trace on the next page) is that when we have this Gleam interpreter print out the `eval/apply` trace, if an `apply` call ends up taking several more steps before finding the result, it will re-print the earlier `apply` call immediately once it's found the result. We can see this with the call to `apply f to [1, 2]`, it prints this and then once it's done applying, it prints `apply f to [1, 2] → [1, 2, 3]`. We don't count this as an additional call to `apply`; it just printed this again to demonstrate that it found the result. Conversely, the call `apply + to 1, 1 → 2` was only printed once, because it immediately found the result and didn't need to have any additional calls to `eval/apply` before finding the result. Ultimately, if we're looking at this type of trace, we have to make sure we disregard any repeated printed calls to `apply` in order to come to the correct number.

```
//Here's the code we're going to run:
fn f(s) {
    case s {
        [] -> [3]
        [first, ..rest] -> [first, ..f(rest)]
    }
}
f([1, 1 + 1])

//And here's the corresponding series of calls to Eval and Apply:
eval f([1, 1 + 1])
  eval f
  eval [1, 1 + 1]
    eval 1
    eval 1 + 1
      eval +
      eval 1
      eval 1
      apply + to 1, 1 → 2
  apply f to [1, 2]
    eval case s { [] -> [3] [first, ..rest] -> ... }
      eval s
    eval [first, ..f(rest)]
      eval first
      eval f(rest)
        eval f
        eval rest
        apply f to [2]
          eval case s { [] -> [3] [first, ..rest] -> ... }
            eval s
          eval [first, ..f(rest)]
            eval first
            eval f(rest)
              eval f
              eval rest
              apply f to []
                eval case s { [] -> [3] [first, ..rest] -> ... }
                  eval s
                eval [3]
                  eval 3
              apply f to [] → [3]
        apply f to [2] → [2, 3]
  apply f to [1, 2] → [1, 2, 3] // not a new apply (see previous page)
result: [1, 2, 3]
```

1. Identify the number of calls to `eval` and the number of calls to `apply` in `evaluator.gleam` made when `run` calls `draw` in each program below. (`run` evaluates the body of `draw` directly, so `draw` itself is not an `apply`.) For each program, count the calls made while evaluating that program, then choose one number from each row of the table. The numbers in the table are answer choices, not calls to add together.

   (a)
   ```
   fn draw(shape_slider: Int) -> Int {
     case True {
       True -> abs(3)
       False -> abs(1)
     }
   }
   ```

eval	4	5	7	9
apply	1	2	3	4

   5 eval, 1 apply

   ```
   eval case True { True -> abs(3) False -> abs(1) }
     eval True
   eval abs(3)
     eval abs
     eval 3
     apply abs to 3 → 3

   result: 3
   ```

   (b)
   ```
   fn draw(shape_slider: Int) -> Point {
     case abs(2) {
       2 -> Point(1, 2)
       _ -> Point(0, 0)
     }
   }
   ```

eval	6	8	10	12
apply	1	2	3	4

   8 eval, 2 apply

   ```
   eval case abs(2) { 2 -> Point(1, 2) _ -> Point(0, 0) }
     eval abs(2)
       eval abs
       eval 2
       apply abs to 2 → 2
   eval Point(1, 2)
     eval Point
     eval 1
     eval 2
     apply Point to 1, 2 → Point(1, 2)

   result: Point(1, 2)
   ```

(c)
```
fn twin(x: Int) -> Point {
  Point(x, x)
}

fn draw(shape_slider: Int) -> Point {
  twin(abs(3))
}
```

eval	6	9	11	14
apply	1	2	3	4

9 eval, 3 apply

```
eval twin(abs(3))
  eval twin
  eval abs(3)
    eval abs
    eval 3
    apply abs to 3 → 3
  apply twin to 3
    eval Point(x, x)
      eval Point
      eval x
      eval x
      apply Point to 3, 3 → Point(3, 3)
  apply twin to 3 → Point(3, 3) // same call as apply twin to 3

result: Point(3, 3)
```

(d)
```
fn pair(x: Int, y: Int) -> Point {
  Point(x, y)
}

fn draw(shape_slider: Int) -> Point {
  pair(abs(3), case True {
    True -> 2
    False -> 0
  })
}
```

eval	10	12	13	15
apply	1	2	3	4

12 `eval`, 3 `apply`

```
eval pair(abs(3), case True { True -> 2 False -> 0 })
  eval pair
  eval abs(3)
    eval abs
    eval 3
    apply abs to 3 → 3
  eval case True { True -> 2 False -> 0 }
    eval True
  eval 2
  apply pair to 3, 2
    eval Point(x, y)
      eval Point
      eval x
      eval y
      apply Point to 3, 2 → Point(3, 2)
  apply pair to 3, 2 → Point(3, 2) // same call as apply pair to 3, 2

result: Point(3, 2)
```

Teaching Tips

- This has historically been a tricky concept for students. `apply` may come off as easier to understand so relate it to just applying functions to already-evaluated arguments for students.
- Remind students of what gets `eval`'ed: every `Call` (the whole call, its function expression, and each operand), variables, literals, `case` subjects and the chosen arm's body, and `let` values and bodies.
- The parser also wraps the body of every function (and each `let` value and each `case` arm's body) in an `At` marker, which technically costs one `eval`. The trace tool at `https://composingprograms.com/3ed/eval/` hides these, and the solutions above (which are its traces) don't count them, so don't have students count them either.
- Operators are ordinary calls: `1 + 2` is a `Call` of the built-in `+`, so it costs 4 `eval`s (the call, `+`, `1`, `2`) and 1 `apply`. Record constructors like `Point` are built-ins too.
- Every `Call` evaluates all of its operands. Only `case` skips work, by evaluating just the arm whose pattern matches (patterns are matched, not evaluated). `&&` and `||` are rewritten by the parser into a `case`, which is how they short-circuit.

- For a `Builtin`, `apply` just calls its `run` function. A `UserFn` `eval`s its body in a new frame. Higher-order built-ins like `list.map` call `apply` on the function they are given, once per element.
- Be very conscious about not accidentally evaluating the expressions yourself when you are counting; that's the interpreter's job!
- It might help to count the expressions by `apply` groups, i.e. count all of `eval` for one `apply`able group and then move onto the next.

The questions in this section use the arithmetic expression language that you saw in the homework, but they do not require finishing the homework.

Expressions

An arithmetic expression is either a variable, an integer, or one of the operators `+`, `-`, `*`, or `/` applied to two smaller expressions. Each case is a variant of `Expr`:

```
pub type Operator {
  Add
  Sub
  Mul
  Div
}

pub type Expr {
  Var(String)
  N(Int)
  Binary(operator: Operator, left: Expr, right: Expr)
}
```

For example, `(x * (2 + 3))` is represented as

```
Binary(Mul, Var("x"), Binary(Add, N(2), N(3)))
```

Note that `Binary` lists its operator *before* its left and right expressions. A variable's name is a single letter from `a` to `z`. In this section, we often write an expression like `(x * (2 + 3))` as shorthand for its `Expr`, with every `Binary` expression in parentheses.

Two helper functions are provided in the homework:

- `symbol(operator: Operator)-> String` returns the symbol for an operator, such as `"+"` for `Add`.
- `calculate(operator: Operator, a: Int, b: Int)-> Int` applies an operator to two integers, so `calculate(Mul, 2, 3)` is `6`. As in Gleam, `/` drops any remainder, and dividing by zero gives `0`.

```
pub fn symbol(operator: Operator) -> String {
  case operator {
    Add -> "+"
    Sub -> "-"
    Mul -> "*"
    Div -> "/"
  }
}

pub fn calculate(operator: Operator, a: Int, b: Int) -> Int {
  case operator {
    Add -> a + b
    Sub -> a - b
    Mul -> a * b
    Div -> a / b
  }
}
```

Functions

A function such as `fn(x, y){ 2 * y - x }` has a list of parameters and a body expression, which a `UserFn` holds:

```
pub type UserFn {
  UserFn(params: List(String), body: Expr)
}
```

For example, `fn(x, y){ 2 * y - x }` is `UserFn(["x", "y"], Binary(Sub, Binary(Mul, N(2), Var("y")), Var("x")))`. A function is *valid* if no parameter is repeated and every variable in its body is one of its parameters. You may assume every `UserFn` passed to a function in this section is valid.

Errors are represented by an `EvalError`. In this section, we only use `WrongArgumentCount`, which records how many arguments a function wants and how many it got:

```
pub type EvalError {
  NameNotFound(String)
  WrongArgumentCount(want: Int, got: Int)
}
```

Homework Functions

For reference, the homework asks you to implement the following functions. The questions in this section don't call them, but they are a good way to think about what each question is doing.

- `to_string(expr)` returns an expression written out as a string, with every `Binary` expression in parentheses. For example, the `Expr` above becomes `"(x * (2 + 3))"`.
- `eval(expr, bindings)` returns the value of an expression when each variable has the value it is bound to in the `Dict` `bindings`, or a `NameNotFound` error for a variable that isn't bound.
- `make_fn(params, body)` returns a `UserFn` if the function is valid, or an error listing the repeated parameters or the variables in the body that aren't parameters.
- `apply(f, args)` returns the value of the body of `f` when each parameter is bound to the argument in the same position, or a `WrongArgumentCount` error if the number of arguments is wrong. For example, applying `fn(x, y){ 2 * y - x }` to `1` and `5` gives `9`.

2. Implement `simplify`, which takes in an `Expr expr` and returns an equivalent expression in which every `Binary` expression whose two sides both simplify to numbers is replaced by the single number it computes. Parts of `expr` that contain variables are kept. For example, `(x + (2 * 3))` simplifies to `(x + 6)`. Use the provided `calculate` function.

```
pub fn simplify(expr: Expr) -> Expr {
  case expr {
    Binary(operator, left, right) ->
      case _____________(__________), _____________(__________) {
        N(a), N(b) -> ____________________________________
        l, r -> ____________________________________
      }
    _ -> _____________
  }
}

assert simplify(Binary(Add, Var("x"), Binary(Mul, N(2), N(3))))
  == Binary(Add, Var("x"), N(6))
assert simplify(Binary(Mul, Binary(Sub, N(5), N(1)), N(2))) == N(8)
assert simplify(Binary(Add, Binary(Add, Var("x"), N(1)), Binary(Add, N(2),
   N(3))))
  == Binary(Add, Binary(Add, Var("x"), N(1)), N(5))
assert simplify(Var("x")) == Var("x")
```

```
pub fn simplify(expr: Expr) -> Expr {
  case expr {
    Binary(operator, left, right) ->
      case simplify(left), simplify(right) {
        N(a), N(b) -> N(calculate(operator, a, b))
        l, r -> Binary(operator, l, r)
      }
    _ -> expr
  }
}
```

We simplify both sides first and then match on the two results at once. If both sides simplified all the way down to numbers, we can compute the result with `calculate` and return a single `N`. Otherwise, at least one side still has a variable in it, so we rebuild the `Binary` around the simplified sides. A `Var` or an `N` is already as simple as it gets, so it is returned as is.

It is important to match on `simplify(left)` and `simplify(right)` rather than on `left` and `right` directly. In `((5 - 1)* 2)`, the left side is a `Binary`, not an `N`, so it only becomes `N(4)` after it has been simplified. Simplifying the sides first is what lets numbers collapse from the bottom of the tree up.

Note that `simplify` only combines two numbers that are directly joined by an operator. In `((x + 1)+ 2)`, the `1` and the `2` are never on the two sides of the same `Binary`, so the expression stays the same even though it is equivalent to `(x + 3)`.

3. Implement `substitute`, which takes in an `Expr` `expr`, a variable `name`, and an `Expr` `replacement`, and returns `expr` with every occurrence of the variable `name` replaced by `replacement`. All other parts of `expr` stay the same. For example, substituting `N(4)` for `"x"` in `(x + y)` gives `(4 + y)`.

```
pub fn substitute(expr: Expr, name: String, replacement: Expr) -> Expr {
  case expr {
    Var(v) if _____________ -> _____________
    Var(_) | N(_) -> _____________
    Binary(operator, left, right) -> {
      let new_left = _____________(_____________, _____________, _____________)
      let new_right = _____________(_____________, _____________, _____________)
      Binary(_____________, _____________, _____________)
    }
  }
}

assert substitute(Binary(Add, Var("x"), Var("y")), "x", N(4))
  == Binary(Add, N(4), Var("y"))
assert substitute(Binary(Mul, Var("x"), Var("x")), "x", Binary(Sub,
   Var("a"), N(1)))
  == Binary(Mul, Binary(Sub, Var("a"), N(1)), Binary(Sub, Var("a"), N(1)))
assert substitute(Binary(Mul, N(2), Var("y")), "x", N(9))
  == Binary(Mul, N(2), Var("y"))
```

```
pub fn substitute(expr: Expr, name: String, replacement: Expr) -> Expr {
  case expr {
    Var(v) if v == name -> replacement
    Var(_) | N(_) -> expr
    Binary(operator, left, right) -> {
      let new_left = substitute(left, name, replacement)
      let new_right = substitute(right, name, replacement)
      Binary(operator, new_left, new_right)
    }
  }
}
```

There is one case for each variant of `Expr`. The `Binary` case recurses on both sides and rebuilds a `Binary` with the same operator around the two substituted sides, so the result has the same shape as `expr`.

The only place anything changes is a `Var` whose name matches, which we check with a guard. Every other leaf, whether a different variable or a number, is returned as is. Note that `replacement` can be any expression, not just a number: substituting `(a - 1)` for `"x"` in `(x * x)` gives `((a - 1)* (a - 1))`, with a copy of the replacement in each spot.

4. Implement `curry`, which takes in a `UserFn f` and an integer `arg`, and returns a function that takes all but the first parameter of `f`, with `arg` bound to the first parameter. For example, currying `fn(x, y){ y - x }` with `1` gives `fn(y){ y - 1 }`. If `f` has no parameters, return `Error(WrongArgumentCount(want: 0, got: 1))`. You may use `substitute` from the previous question.

```
pub fn curry(f: UserFn, arg: Int) -> Result(UserFn, EvalError) {
  case f.params {
    [] -> Error(WrongArgumentCount(want: 0, got: 1))
    [first, ..rest] -> {
      let new_body = _____________(____________, ____________, ____________)
      Ok(UserFn(____________, ____________))
    }
  }
}

assert curry(UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))), 1)
  == Ok(UserFn(["y"], Binary(Sub, Var("y"), N(1))))
assert curry(UserFn(["c"], Binary(Add, N(10), Var("c"))), 1)
  == Ok(UserFn([], Binary(Add, N(10), N(1))))
assert curry(UserFn([], N(7)), 1) == Error(WrongArgumentCount(want: 0,
   got: 1))
```

```
pub fn curry(f: UserFn, arg: Int) -> Result(UserFn, EvalError) {
  case f.params {
    [] -> Error(WrongArgumentCount(want: 0, got: 1))
    [first, ..rest] -> {
      let new_body = substitute(f.body, first, N(arg))
      Ok(UserFn(rest, new_body))
    }
  }
}
```

A `UserFn` is just a list of parameters and a body, so binding the first parameter means changing both: the new parameter list is everything after the first, and the new body is the old body with the first parameter replaced by the argument. Since the body is an `Expr`, the argument has to be wrapped as `N(arg)` before it can be substituted in.

The expression `f.params` accesses the `params` field of `f`, which works because `UserFn` has only one variant. We could have instead written `let UserFn(params, body) = f` first.

Currying with the first argument and then applying the result to the remaining arguments gives the same value as applying `f` to all of them at once: `apply(f, [1, 4])` and `apply(g, [4])`, where `g` is `curry(f, 1)`, both return `Ok(3)`. This relies on `f` having no repeated parameters, which `make_fn` guarantees; if `f` had parameters `["x", "x"]`, substituting for the first `x` would replace both.

5. Implement `compose`, which takes in two `UserFn`s `f` and `g`, and returns a function `h` such that applying `h` to some arguments gives the same result as applying `g` to those arguments and then applying `f` to that result. For example, composing `fn(x){ x * x }` with `fn(x, y){ y - x }` gives `fn(x, y){ { y - x } * { y - x } }`. Since `f` receives exactly one value, return a `WrongArgumentCount` error if `f` does not take exactly one parameter. You may use `substitute`.

```
pub fn compose(f: UserFn, g: UserFn) -> Result(UserFn, EvalError) {
  case f.params {
    _______ -> {
      let new_body = _____________(____________, ____________, ____________)
      Ok(UserFn(____________, ____________))
    }
    params -> Error(WrongArgumentCount(want: ____________(_______), got: 1))
  }
}

assert compose(
    UserFn(["x"], Binary(Mul, Var("x"), Var("x"))),
    UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))),
  )
  == Ok(UserFn(
    ["x", "y"],
    Binary(Mul, Binary(Sub, Var("y"), Var("x")), Binary(Sub, Var("y"),
        Var("x"))),
  ))
assert compose(UserFn(["x", "y"], Binary(Sub, Var("y"), Var("x"))),
   UserFn(["x"], Var("x")))
  == Error(WrongArgumentCount(want: 2, got: 1))


pub fn compose(f: UserFn, g: UserFn) -> Result(UserFn, EvalError) {
  case f.params {
    [x] -> {
      let new_body = substitute(f.body, x, g.body)
      Ok(UserFn(g.params, new_body))
    }
    params -> Error(WrongArgumentCount(want: list.length(params), got: 1))
  }
}
```

The composed function takes whatever arguments `g` takes, so its parameters are `g.params`. Its body should compute `f`'s body where `f`'s parameter is the value of `g`'s body, so we substitute the whole expression `g.body` for `f`'s parameter. The pattern `[x]` matches only a list with exactly one element, which both checks that `f` has one parameter and gives us its name. In the error, `f` wants `list.length(params)` arguments but would only ever get the 1 value that `g` produces.

Using the same parameter name in both functions is fine. Composing `fn(x){ x * 2 }` with `fn(x){ x + 3 }` gives `fn(x){ { x + 3 } * 2 }`, because `substitute` only makes one pass over `f`'s body and never looks inside the replacement it puts in.

Also note that `g`'s body is copied into every place `f` uses its parameter, so in the `(x * x)` example `(y - x)` is evaluated twice. This gives the right answer because evaluating an expression has no side effects, but it does duplicate work.